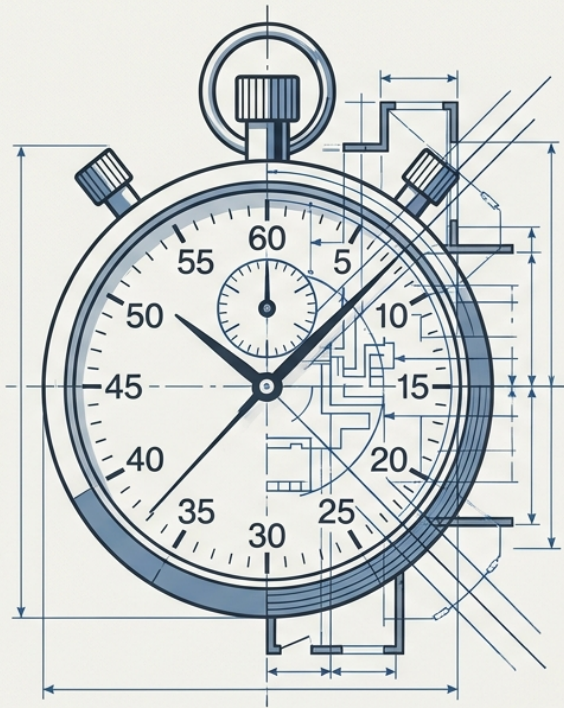


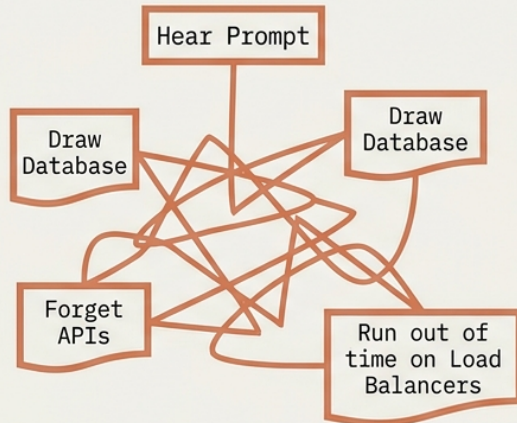


The 45-Minute System Design Playbook

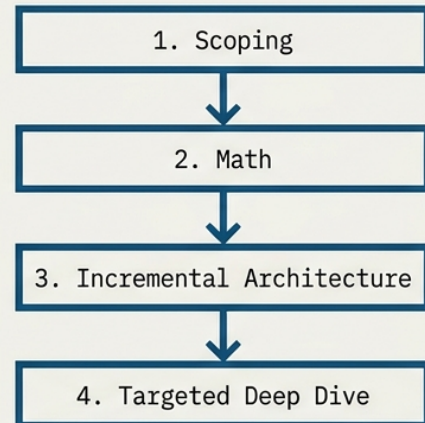
A rigid, repeatable framework for deriving architectures from math, bypassing memorization, and surviving the ticking clock. Featuring a complete Google Docs case study.

The interview evaluates discipline, not just knowledge

Without Structure: The Trap



With Framework: The Pipeline



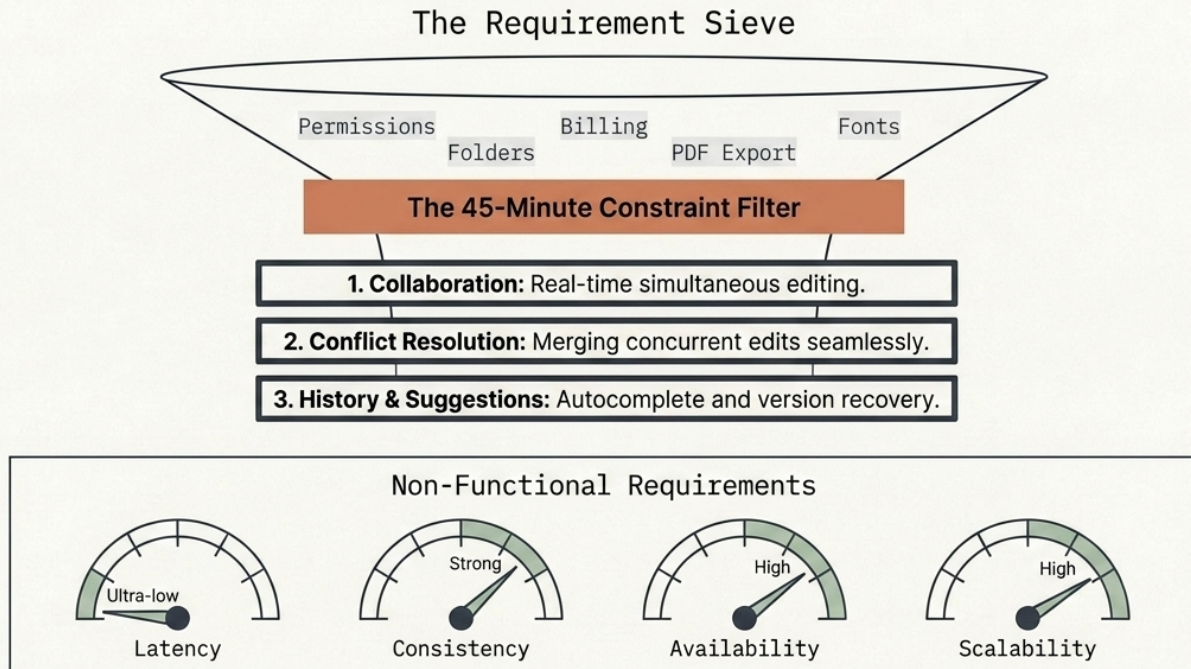
Key Takeaway: The goal is not to memorize the final design of Twitter or Google Docs. The goal is to demonstrate a repeatable mental process that makes the right architecture inevitable.

Master the ticking clock with four strict phases

Phase 1: Requirements [5 min]	Phase 2: Scale Estimation [5 min]	Phase 3: High-Level Design [15 min]	Phase 4: Deep Dive [20 min]
Scoping functional and non-functional boundaries.	Deriving the architectural blueprint from math.	Building incrementally from a single-user baseline.	Trade-offs, bottlenecks, and the core technical challenge.

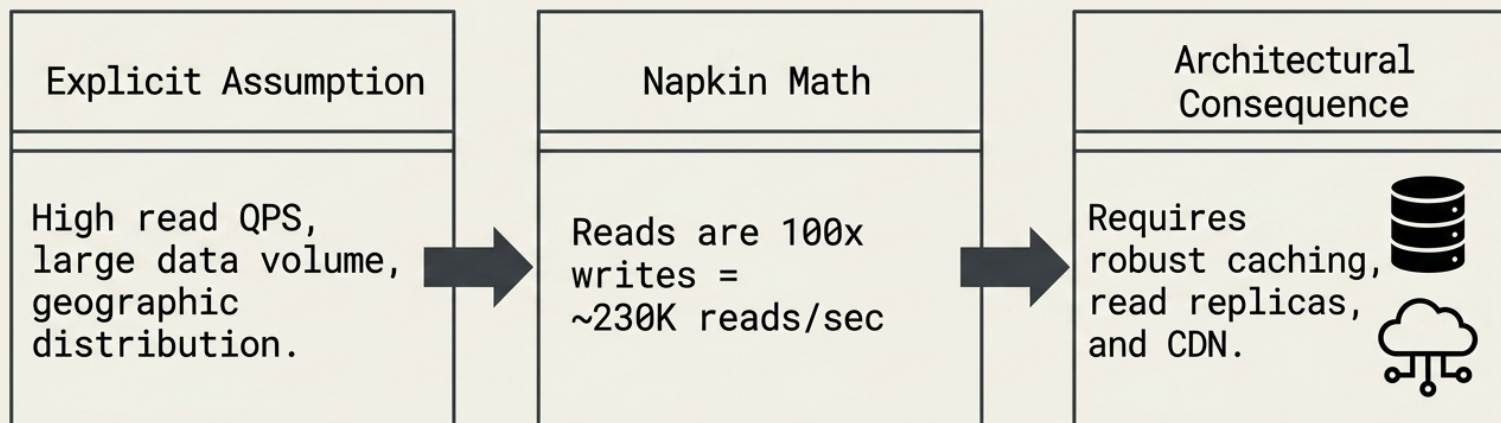
We will prove this framework by building a collaborative document editor (Google Docs) from scratch.

Filter infinite possibilities into actionable boundaries



Do not guess at components; **derive** them from math

The Math-to-Architecture Pipeline



Takeaway: Every component on your final whiteboard must trace its existence back to a number generated during scale estimation.

Execute the estimation pipeline for document collaboration

Storage Architecture

80M DAU × 1 doc/day = 80M docs/day
Text: 100KB, Images (30%): 800KB, Video (2%): 3MB
Result: 32 TB/day total storage



Blob Storage

Bandwidth Architecture

Incoming: 32TB / 86400s ≈ 3 Gbps
Outgoing (5 views/user): ≈ 14.8 Gbps
Result: ~18 Gbps total



Load Balancers / CDN

Compute Architecture

RPS proxy: 80M requests/sec (peak load approximation)
Server capacity: 64,000 RPS
Result: ~1.3K servers at peak



Horizontally Scaled
App Cluster

Anchor the system behavior by defining the core APIs

`ws://api.docs.com/stream/{doc_id}`

WebSockets over HTTP. Essential for low-latency, real-time bi-directional collaboration without overhead.

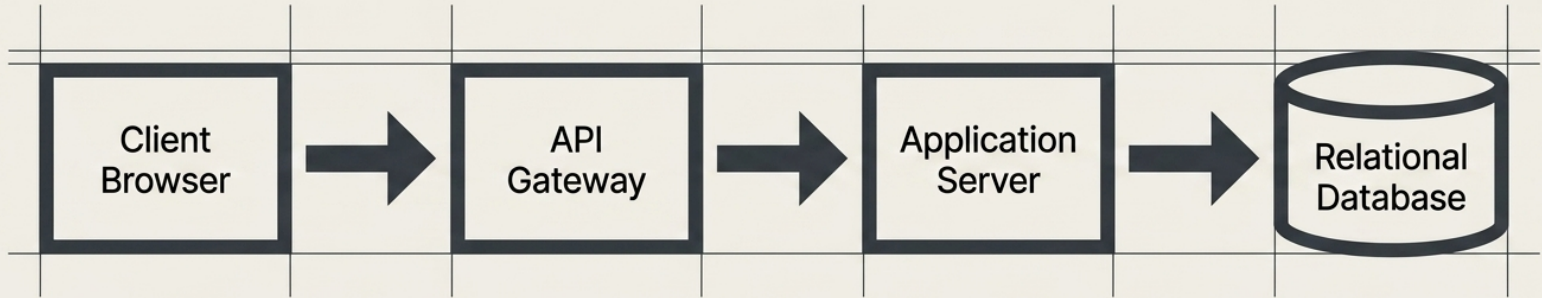
```
{  
  'action': 'insert',  
  'char': 'A',  
  'index': 2,  
  'userId': 'u123'  
}
```

Atomic operation payload. Notice the reliance on positional indexing—this will trigger our primary concurrency challenge.

`GET /docs/{doc_id}`

Returns baseline snapshot to hydrate client before WebSocket takes over.

Build incrementally: Start with the single-user baseline

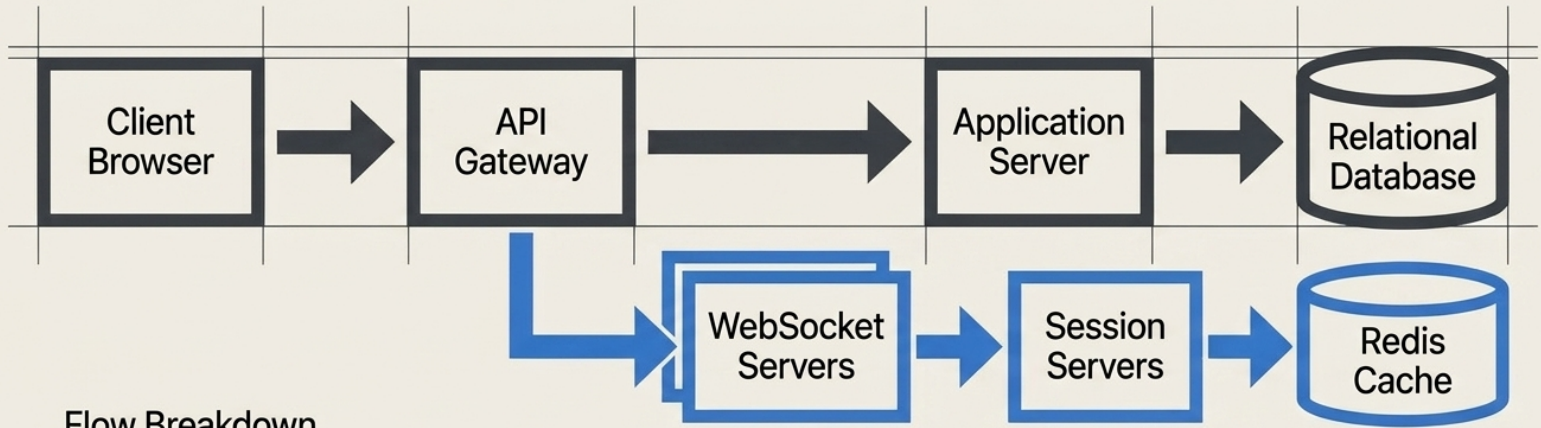


Before introducing distributed complexity, prove the basic write/read path.

A single user creating a document simply writes metadata and text state to a single database.

Why add complexity?
Because this baseline fails instantly if a second user connects or if traffic spikes.

Evolve the architecture to support real-time presence

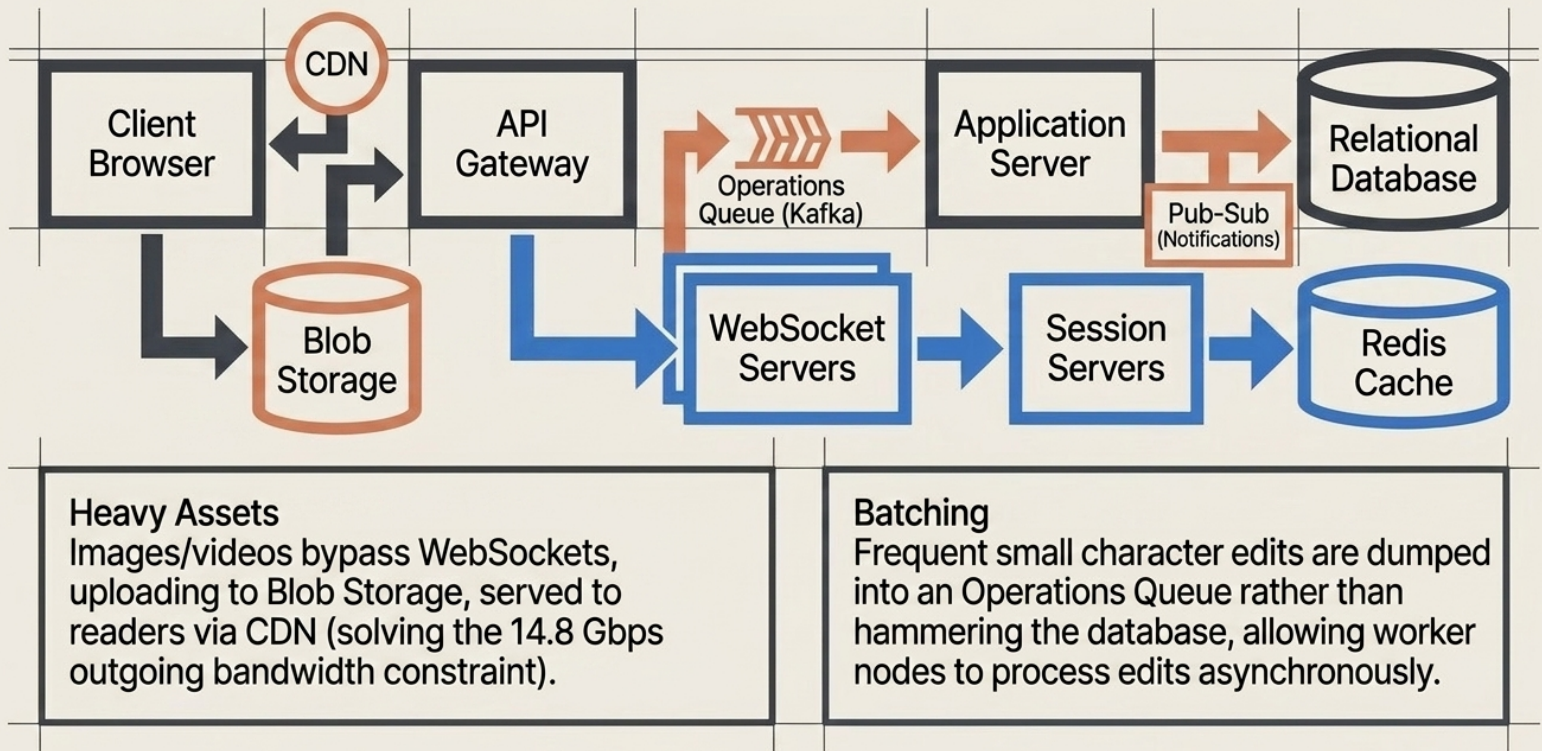


Flow Breakdown

Step 1: Client upgrades HTTP to WebSocket via Gateway.
Step 2: WebSocket Server maintains the persistent connection.
Step 3: Session Server tracks active collaborators in a Redis Cache.

HTTP request/response is too slow for keystroke-level updates. WebSockets provide full-duplex communication, while Session Servers act as the source of truth for "who is currently online".

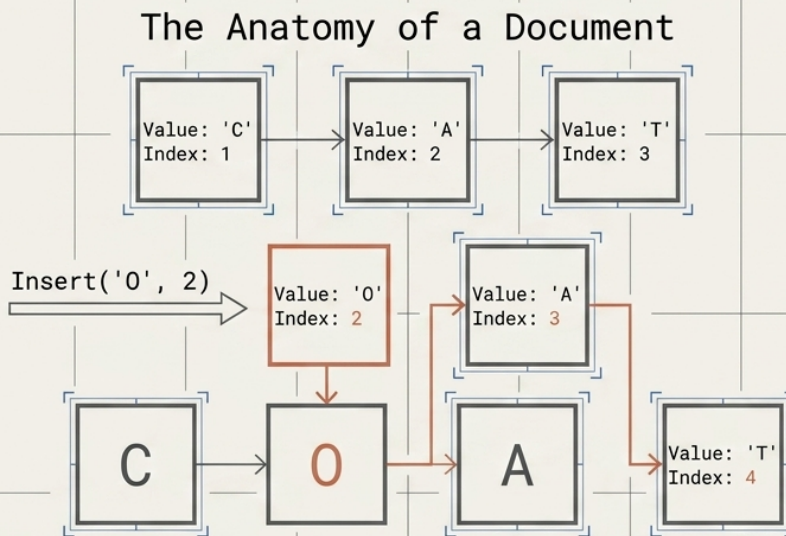
Introduce asynchronous processing and asset delivery for scale



The Database Routing Matrix: Match storage to data topology

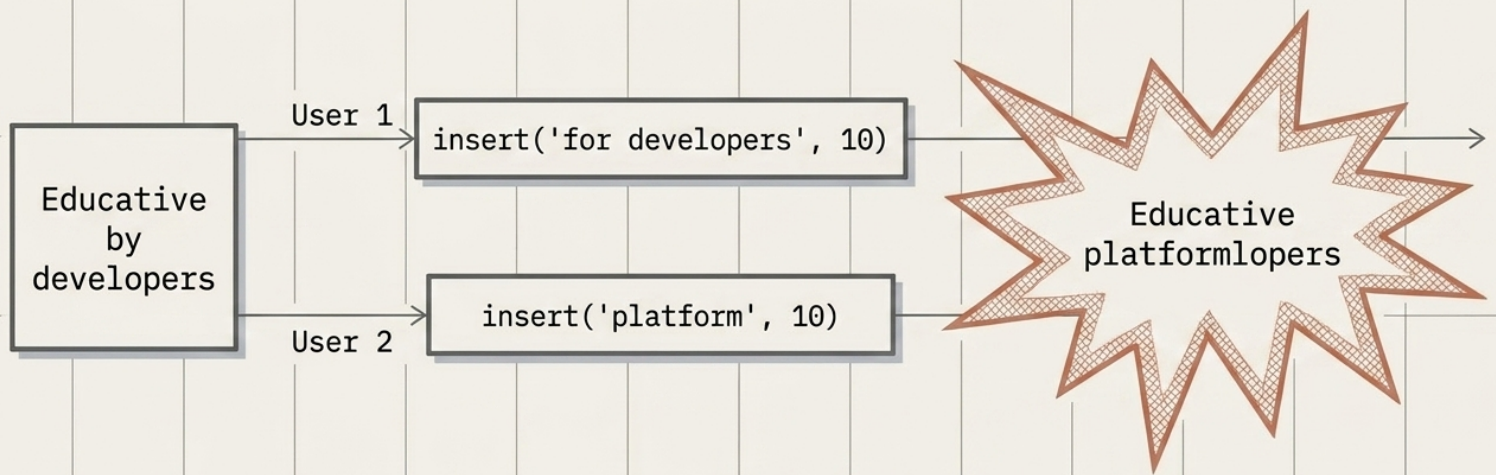
Data Type	Storage Engine	Architectural Justification
User Metadata & Permissions	Relational (PostgreSQL)	ACID compliance, structured relationships.
Document Edit History	Time-Series Database	Append-only sequential tracking of version recovery and DIFF operations.
Comments & Suggestions	NoSQL	Fast retrieval, flexible schema, massive volume of short text.
Media (Images/Video)	Blob Storage + CDN	Large payloads (32TB/day), static assets.

Isolate the most interesting technical challenge



In a standard interview, the Load Balancer is boring. Concurrency is the bottleneck. Because documents rely on positional indexing, simultaneous edits from distributed users will inevitably corrupt the state.

The Divergence-Convergence Failure Model



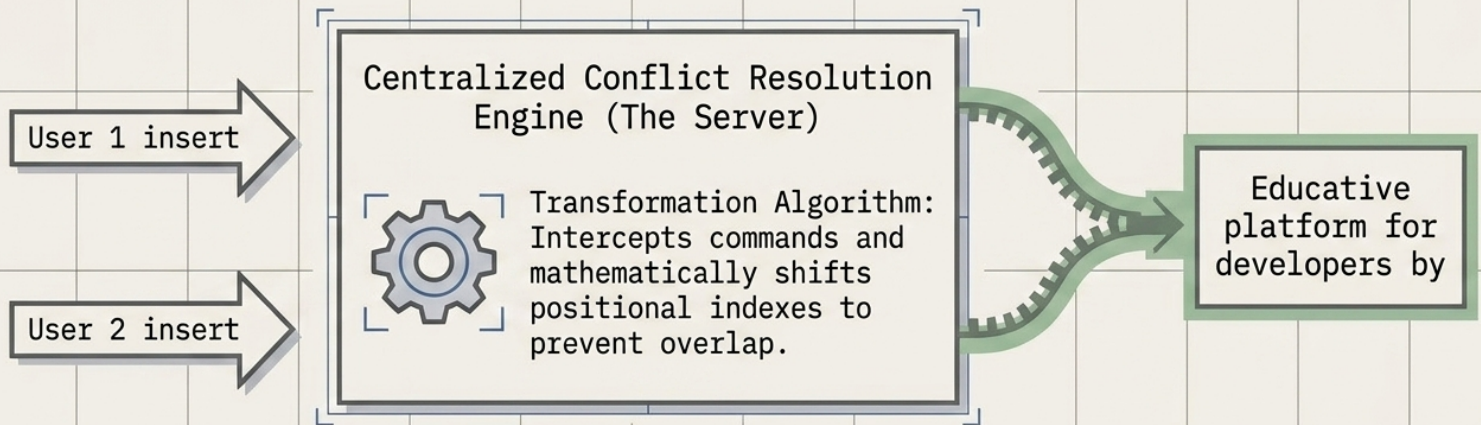
Commutativity Failure

Operations applied in different orders yield different, corrupted results.

Idempotency Failure

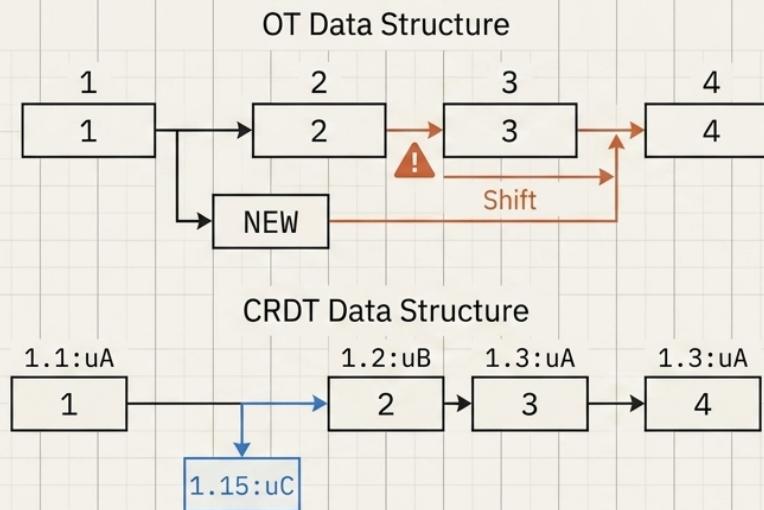
Applying the same operation twice (e.g., both users delete a character) executes twice instead of once.

The Concurrency Engine: Operational Transformation (OT)



OT guarantees Convergence (CC consistency model)—all replicas eventually become identical. However, it requires complex centralized logic.

The Decentralized Alternative: Conflict-free Replicated Data Types (CRDT)



CRDTs abandon order-dependency entirely. By assigning a globally unique identity and a fractional global order to every character, insertions never alter the positional index of existing characters.

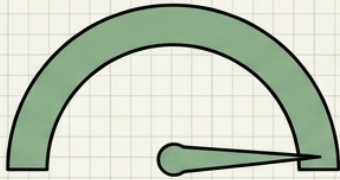
Architectural Consequence:

CRDTs guarantee eventual consistency and allow for serverless or Peer-to-Peer (P2P) collaboration, removing the need for a central conflict resolution engine.

The Concurrency Matrix: OT vs. CRDT

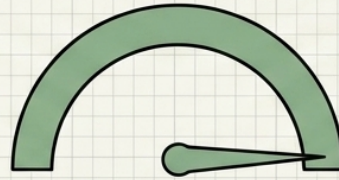
Dimension	Operational Transformation (OT)	CRDT
Mechanism	Transforms operations centrally	Complex localized data structures
Index Strategy	Absolute Integer	Fractional / Relative
Architecture	Client-Server / Centralized	P2P / Serverless compatible
Implementation	Notoriously complex algorithms	High memory overhead, easier to reason about
Real-World	Google Docs, Etherpad	Figma, Modern decentralized apps

Closing the loop: Non-Functional Requirements Compliance



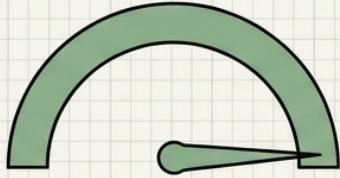
LATENCY

Achieved via WebSocket bidirectional streams and CDN offloading for 32TB of media.



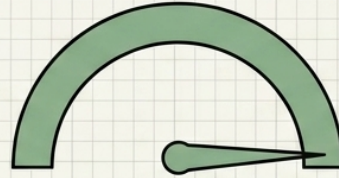
CONSISTENCY

Achieved via OT algorithms maintaining the CC consistency model and Time-Series DBs for operation ordering.



AVAILABILITY

Achieved via internal Replication (Gossip protocol) across data centers and redundant WebSocket servers.



SCALABILITY

Achieved via horizontal sharding of RDBMS and dedicated Operation Queues per active document.

The Architecture is the Math

